



**MANIPAL UNIVERSITY
JAIPUR**

A Report

on

GPU ALLOCATION SYSTEM

carried out as part of the course Project Based Learning-4

Submitted by

ADHYA SHARMA

23FE10CSE00166

VI Semester

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering



**MANIPAL UNIVERSITY
JAIPUR**

Department of Computer Science & Engineering,

School of Computer Science and Engineering,

Manipal University Jaipur,

Jan-May 2026



**MANIPAL UNIVERSITY
JAIPUR**

(University under Section 2(f) of the UGC Act)

Department of Computer Science and Engineering

School of Computer Science and Engineering

Date: 13-04-2026

CERTIFICATE

This is to certify that the project entitled "GPU Allocation System" is a bonafide work carried out as PBL-4 End Term Assessment in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, by Adhya Sharma bearing registration number (23FE10CSE00166), during the academic semester VI of year 2025-2026.

Place: Manipal University Jaipur, Jaipur

Name of the project guide: Mr. Jay Shankar Sharma

Signature of the project guide: _____

Under the Guidance of:

Guide Name: Mr . Jay Shankar Sharma

Guide Signature (with date):

Acknowledgement

This project would not have been completed without the help, support, comments, advice, cooperation and coordination of various people. However, it is impossible to thank everyone individually; I am hereby making a humble effort to thank some of them.

I acknowledge and express my deepest sense of gratitude to my internal supervisor for his/her constant support, guidance, and continuous engagement. I highly appreciate the technical comments, suggestions, and criticism during the progress of this project titled "GPU Allocation System".

I owe my profound gratitude to Dr. Neha Chaudhary, Head, Department of CSE, for her valuable guidance and for facilitating me during my work. I am also very grateful to all the faculty members and staff for their precious support and cooperation during the development of this project.

The GPU Allocation System was a challenging yet deeply rewarding undertaking. Building a full-stack MERN application with real-time scheduling, role-based access control, and Docker containerization required extensive research, iterative development, and problem-solving. This would not have been possible without the constant encouragement of my peers and mentors.

Finally, I extend my heartfelt appreciation to my classmates for their help and encouragement.

Adhya Sharma
23FE10CSE00166

ABSTRACT

The GPU Allocation System is a full-stack web application designed to address the critical challenge of fair and efficient GPU resource management in shared academic computing environments such as university laboratories and research clusters. In modern educational institutions, GPU resources are limited and highly contested, often leading to resource monopolization, inequitable access, and idle periods that reduce overall utilization efficiency.

This system provides a comprehensive solution built on the MERN stack (MongoDB, Express.js, React, Node.js) with TypeScript on the frontend, containerized using Docker for seamless deployment. The application implements role-based access control for three user categories: students, faculty, and administrators. Students can submit GPU allocation requests specifying purpose, duration, priority level, and preferred time shifts. Administrators review, approve, or reject these requests, with the system automatically allocating specific GPU units upon approval.

Key technical features include priority-based scheduling with low, medium, and high priority levels, automated resource release via Node-cron jobs, real-time utilization monitoring dashboards, a complete audit trail of allocation logs, and an integrated chat system for student-administrator communication regarding extension requests. The system enforces institutional email domain restrictions (@muj.manipal.edu) and employs session-based authentication.

The architecture follows modern software engineering principles including containerized microservices, RESTful API design, environment-based configuration, and separation of concerns between frontend and backend. The deployed system operates with an Nginx reverse proxy serving the React frontend and a Node.js backend connecting to a MongoDB database instance.

Results demonstrate the system effectively prevents resource monopolization, provides transparency in allocation decisions through real-time dashboards, and reduces administrative overhead through automated lifecycle management of GPU allocations. Future enhancements include LDAP/SSO integration, Kubernetes orchestration for cluster environments, and cloud GPU provider integrations.

Keywords: GPU Resource Allocation, MERN Stack, Role-Based Access Control, Docker, Real-Time Monitoring, Priority Scheduling, MongoDB, Node.js, React, TypeScript

TABLE OF CONTENTS

1. Introduction.....	7
1.1 Objective of the Project	7
1.2 Brief Description of the Project	7
1.3 Technology Used	8
1.3.1 Hardware Requirements.....	8
1.3.2 Software Requirements	8
1.4 Organization Profile.....	9
2. Design Description.....	10
2.1 System Architecture.....	10
2.2 Flow Chart	11
2.3 Data Flow Diagrams (DFD).....	12
2.3.1 Level 0 DFD (Context Diagram)	12
2.3.2 Level 1 DFD	12
2.4 Entity Relationship Diagram (E-R Diagram)	13
3. Project Description.....	14
3.1 Database.....	14
3.2 Table Description (Collection Schemas)	14
3.2.1 User Collection	14
3.2.2 GPU Collection.....	14
3.2.3 Request Collection.....	15
3.3 File/Database Design	15
4. Input/Output Form Design.....	16
4.1 Login Page	16
4.2 Student Dashboard	16
4.3 GPU Request Submission Form	16
4.4 Admin Dashboard	16
4.5 Request Management Interface.....	17
4.6 Chat/Extension Request Interface.....	17
5. Testing & Tools Used	17
5.1 Testing Strategy	17
5.2 Unit Testing	18
5.3 API Integration Testing.....	18

5.4 Test Cases	18
5.5 Tools Used for Testing.....	19
6. Implementation & Maintenance	19
6.1 Backend Implementation	19
6.2 Frontend Implementation.....	19
6.3 Deployment Implementation	20
6.4 API Endpoint Structure.....	20
6.5 Maintenance Considerations	21
7. Conclusion and Future Work	21
7.1 Conclusion	21
7.2 Future Work.....	22
8. Outcome.....	23
9. Bibliography	24

1. Introduction

1.1 Objective of the Project

The primary objective of the GPU Allocation System is to design and implement a robust, fair, and transparent platform for managing GPU computational resources in shared academic environments. The specific objectives are:

- To develop a web-based platform enabling students to submit GPU resource requests with priority-based scheduling to ensure fair access.
- To implement real-time GPU utilization monitoring and reporting dashboards for administrators and students.
- To automate the GPU lifecycle management including allocation, monitoring, extension handling, and release through scheduled background jobs.
- To enforce institutional access policies through email domain validation, role-based access control, and session management.
- To deploy the entire system as containerized microservices using Docker for portability, scalability, and ease of maintenance.
- To provide a communication channel between students and administrators for handling special requests such as time extensions.

1.2 Brief Description of the Project

The GPU Allocation System is a full-stack MERN web application that manages GPU resource allocation in shared computing environments such as college labs and research clusters. The system ensures fair distribution, real-time monitoring, and priority-based scheduling to prevent resource monopolization.

The application supports three user roles: Students can submit GPU requests specifying their computing needs, administrators manage and approve allocations while monitoring system-wide GPU health, and the system itself acts autonomously through scheduled cron jobs to release expired allocations.

The frontend is built with React 18 and TypeScript, providing a responsive interface with separate dashboards for students and administrators. The backend is a Node.js/Express.js RESTful API server connected to a MongoDB database via Mongoose ODM. The system is fully containerized with Docker, enabling consistent deployment across environments.

Key operational workflows include: GPU request submission and approval, automated resource release, extension request handling, real-time utilization visualization, and a complete audit log system. The platform restricts registration to Manipal University Jaipur institutional email addresses, ensuring only authorized users gain access.

1.3 Technology Used

1.3.1 Hardware Requirements

Table 1: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 / AMD Ryzen 5	Intel Core i7 / AMD Ryzen 7 or higher
RAM	8 GB	16 GB or higher
Storage	50 GB HDD	100 GB SSD
Network	100 Mbps LAN	Gigabit Ethernet
GPU (Server)	Any CUDA-compatible GPU	NVIDIA RTX 3080 or higher
Display	1280 x 720	1920 x 1080 or higher

1.3.2 Software Requirements

Table 2: Software Requirements

Software/Technology	Version	Purpose
Node.js	18 (Alpine Linux)	Backend runtime environment
Express.js	v5.1.0	Web framework for backend API
MongoDB	6.x	NoSQL database for persistent storage
Mongoose ODM	v8.19.1	MongoDB object modelling for Node.js
React	18.3.1	Frontend UI library
TypeScript	5.5.3	Typed superset of JavaScript
Vite	v5.4.2	Frontend build tool

Tailwind CSS	v3.4.1	Utility-first CSS framework
Axios	v1.12.2	HTTP client for API requests
React Router DOM	v7.9.4	Client-side routing
node-cron	v3.0.3	Scheduled task execution
Docker	Latest	Containerization and deployment
Nginx	Alpine	Frontend static file serving & reverse proxy
dotenv	v17.2.3	Environment variable management
CORS	v2.8.5	Cross-Origin Resource Sharing handler

1.4 Organization Profile

Manipal University Jaipur (MUJ) is a private deemed-to-be university located in Jaipur, Rajasthan, India. Established under Section 3 of the UGC Act, MUJ is part of the Manipal Education and Medical Group (MEMG), one of India's leading educational conglomerates.

The Department of Computer Science and Engineering at MUJ houses multiple computer laboratories equipped with high-performance workstations including GPU-enabled machines used for deep learning, computer vision, scientific computing, and graphics research. The proliferation of AI/ML coursework has significantly increased demand for GPU resources, creating the need for a systematic allocation and management platform.

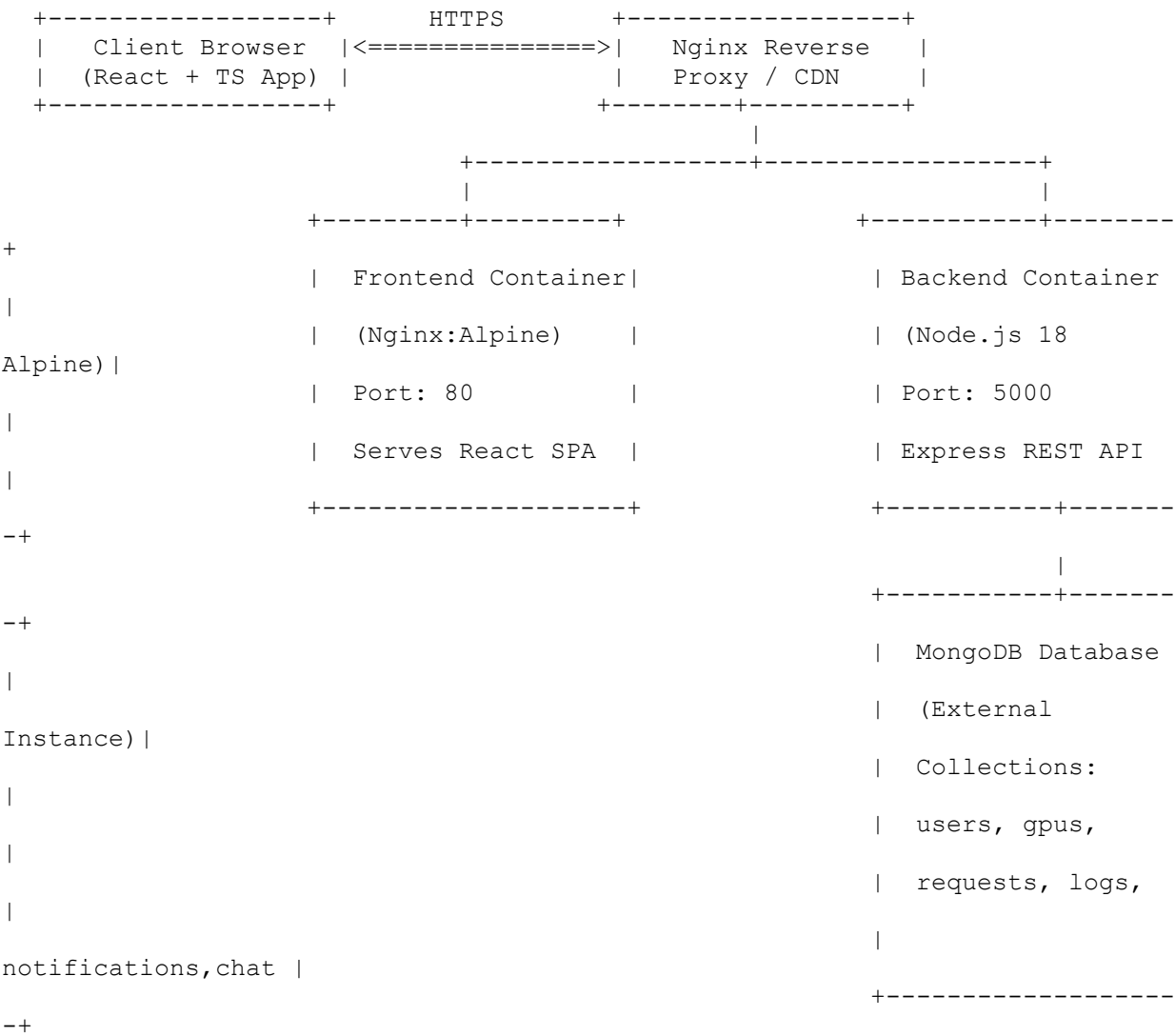
The GPU Allocation System was developed to serve the needs of students and faculty in this academic environment, with the system configured specifically for the MUJ institutional domain (@mu.j.manipal.edu), ensuring that only registered university members can access and utilize the platform.

2. Design Description

2.1 System Architecture

The GPU Allocation System follows a three-tier architecture with clear separation between the presentation layer (React frontend), business logic layer (Node.js/Express backend), and data layer (MongoDB). The system is containerized using Docker, with separate containers for the frontend and backend, and an external MongoDB instance.

Figure 1: High-Level System Architecture



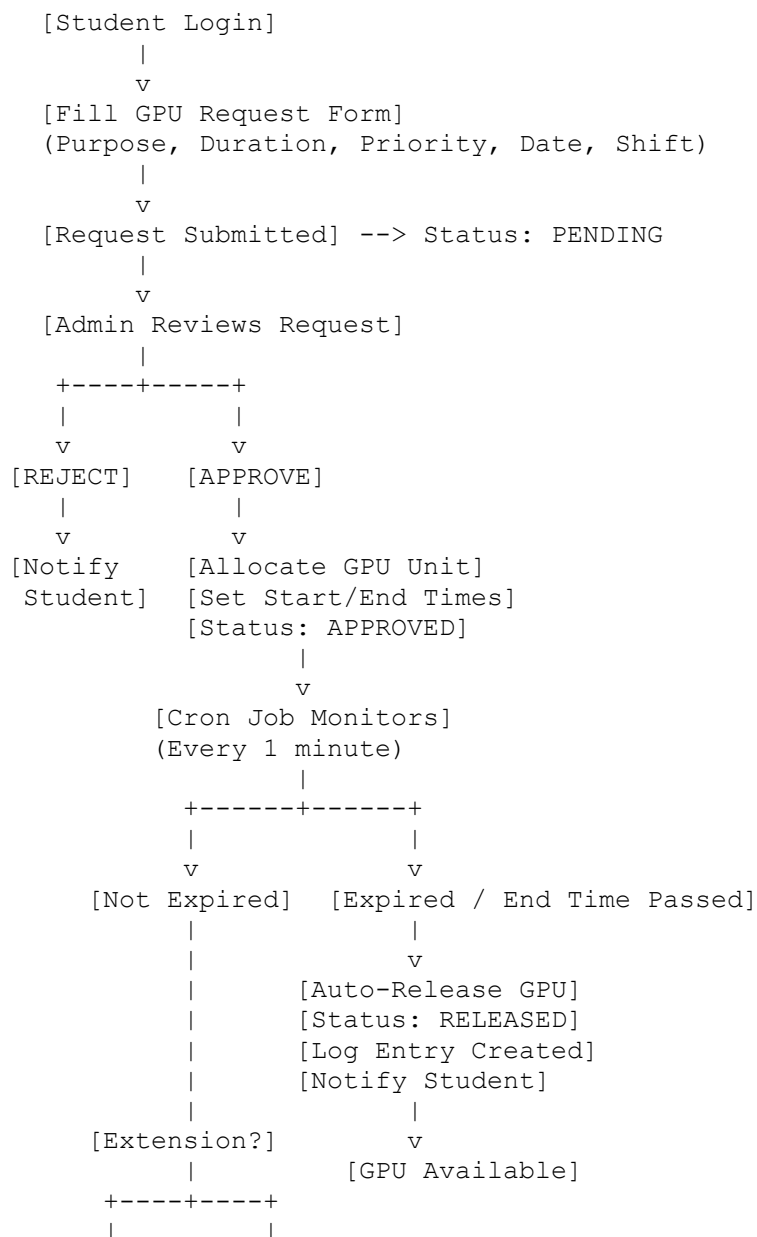
The architecture ensures scalability and maintainability. The Nginx container serves the compiled React SPA and proxies API requests to the Node.js backend. The backend communicates with

MongoDB through Mongoose ODM. Environment variables manage configuration across environments, and Docker Compose orchestrates multi-container deployment.

2.2 Flow Chart

The following flowchart illustrates the complete GPU request lifecycle from student submission to automated resource release.

Figure 2: GPU Request Lifecycle Flowchart



```

      v      v
[No Ext]  [Student Requests Extension]
          [Admin Reviews]
          [Update End Time if Approved]

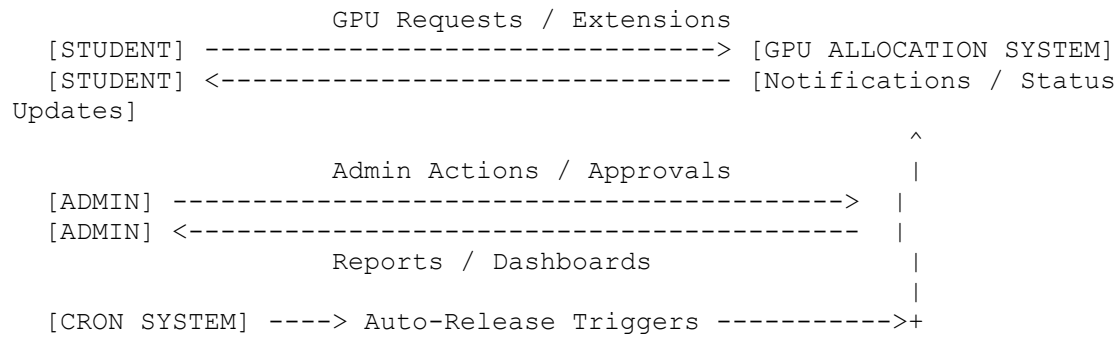
```

2.3 Data Flow Diagrams (DFD)

2.3.1 Level 0 DFD (Context Diagram)

The Level 0 DFD shows the GPU Allocation System as a single process interacting with external entities.

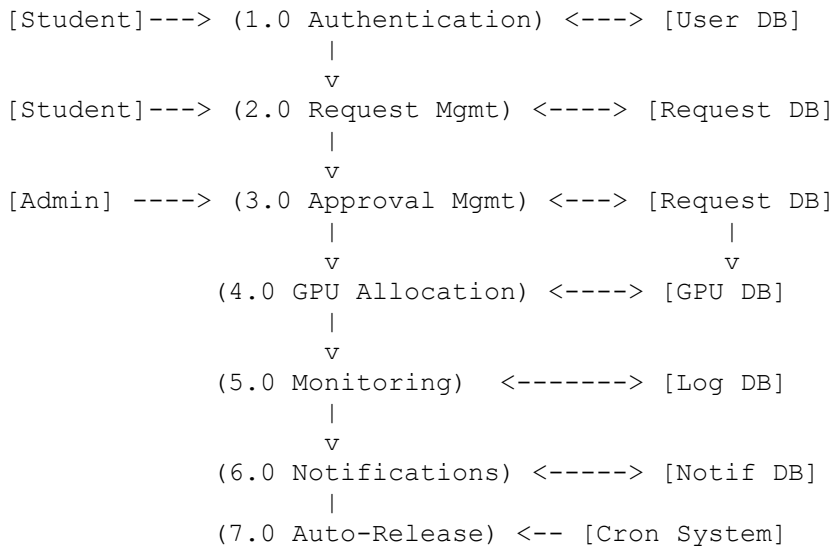
Figure 3: Level 0 DFD - Context Diagram



2.3.2 Level 1 DFD

The Level 1 DFD decomposes the system into its primary processes.

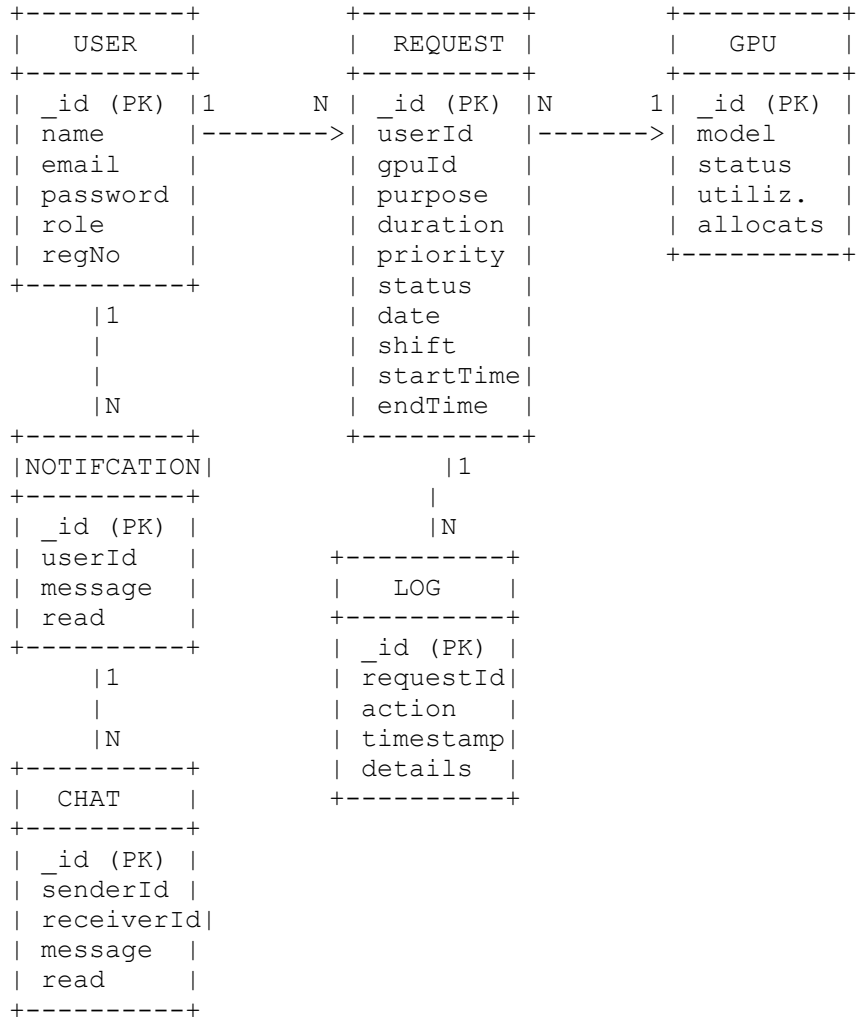
Figure 4: Level 1 DFD



2.4 Entity Relationship Diagram (E-R Diagram)

The following ER diagram describes the data entities and their relationships within the GPU Allocation System.

Figure 5: Entity Relationship Diagram



3. Project Description

3.1 Database

The GPU Allocation System uses MongoDB, a document-oriented NoSQL database, as its primary data store. MongoDB was chosen for its flexible schema design, horizontal scalability, and seamless integration with the Node.js ecosystem through the Mongoose ODM. The database stores all application data across six primary collections: Users, GPUs, Requests, Logs, Notifications, and Chat messages.

The database connection is managed via environment variables (MONGO_URI), allowing easy switching between development, testing, and production database instances. Mongoose provides schema validation, middleware hooks, and a rich query API that enforces data integrity at the application level.

3.2 Table Description (Collection Schemas)

3.2.1 User Collection

Table 3: User Collection Schema

Field	Type	Constraints	Description
_id	ObjectId	PK, Auto-generated	Unique user identifier
name	String	Required	Full name of the user
email	String	Required, Unique, @muj.manipal.edu	Institutional email address
password	String	Required, Hashed	Encrypted user password
role	String	Enum: student/admin	User role for access control
regNo	String	Optional	Student registration number
createdAt	Date	Auto-generated	Account creation timestamp

3.2.2 GPU Collection

Table 4: GPU Collection Schema

Field	Type	Constraints	Description
_id	ObjectId	PK, Auto-generated	Unique GPU identifier
model	String	Required	GPU model name (e.g., RTX 3080)

status	String	Enum: active/idle/maintenance	Current operational state
utilization	Number	0-100	Current utilization percentage
allocations	Array	Ref: Request	Array of active allocation references
updatedAt	Date	Auto-generated	Last update timestamp

3.2.3 Request Collection

Table 5: Request Collection Schema

Field	Type	Constraints	Description
_id	ObjectId	PK, Auto-generated	Unique request identifier
userId	ObjectId	Required, Ref: User	Requesting student reference
gpuId	ObjectId	Ref: GPU	Assigned GPU reference (post-approval)
purpose	String	Required	Description of computing task
duration	Number	Required, in hours	Requested allocation duration
priority	String	Enum: low/medium/high	Request priority level
status	String	Enum: pending/approved/rejected/released/cancelled	Request lifecycle state
date	Date	Required	Requested allocation date
shift	String	Enum: morning/evening	Time shift preference
startTime	Date	Set on approval	Actual allocation start time
endTime	Date	Set on approval	Computed allocation end time

3.3 File/Database Design

The application follows a Model-View-Controller (MVC) architectural pattern for the backend. Database models are defined using Mongoose schemas in dedicated model files, controllers handle business logic, and routes map HTTP endpoints to controller functions.

The frontend state management uses React's built-in `useState` and `useEffect` hooks, with `Axios` handling all HTTP communication with the backend API. TypeScript interfaces mirror the MongoDB schema structures, ensuring type safety across the full stack.

Configuration is managed through environment variables loaded via `dotenv`. Separate `.env` files exist for development and production environments, controlling database URIs, API base URLs,

port configurations, and feature flags such as USE_DUMMY_USERS for development without a live backend.

4. Input/Output Form Design

4.1 Login Page

The login form accepts institutional email (@muj.manipal.edu) and password. Input validation enforces the email domain restriction client-side before submission. Error messages display inline for invalid credentials or unauthorized email domains.

4.2 Student Dashboard

The student dashboard presents a personalized overview including request statistics (pending, approved, rejected, released counts), a GPU utilization chart showing current resource availability, and a notification panel displaying recent system updates. A prominent button navigates to the request submission form.

4.3 GPU Request Submission Form

The request form collects the following inputs:

- Purpose (text area): Detailed description of the computing task (e.g., training a neural network model)
- Duration (numeric input): Number of hours required, validated for positive values
- Priority (dropdown): Selection of low, medium, or high priority
- Date (date picker): Desired allocation date, restricted to future dates
- Shift (radio/dropdown): Morning shift (9 AM - 3 PM) or Evening shift (3 PM - 9 PM)

4.4 Admin Dashboard

The admin dashboard provides a comprehensive operational view including pending request queue, GPU fleet status overview (active, idle, maintenance states), system-wide utilization metrics, and recent activity logs. Tabbed navigation organizes the interface into Request Management, GPU Monitoring, and Allocation Logs sections.

4.5 Request Management Interface

Administrators see a tabulated list of all requests filterable by status. Each row displays student name, purpose, priority, date, shift, duration, and current status. Action buttons enable approve/reject operations with optional admin notes. Approved requests trigger immediate GPU allocation and student notification.

4.6 Chat/Extension Request Interface

The ChatModal component enables direct messaging between students and administrators. Students can initiate extension requests through the chat, providing additional context. Administrators can view conversation history, respond with decisions, and extension approvals automatically update the allocation end time in the database. Unread message counts appear as badges on the chat icon.

Table 6: System Input/Output Summary

Module	Input	Output/Result
Authentication	Email, Password	Session token, Role-based redirect
GPU Request Form	Purpose, Duration, Priority, Date, Shift	Request record created (PENDING status)
Admin Approval	Request ID, Decision (approve/reject), Notes	Status update, GPU allocation, Notification
Extension Request	Request ID, Additional hours, Justification	Updated end time, Log entry, Notification
GPU Monitoring	GPU ID query	Real-time utilization %, allocation history
Allocation Logs	Date range, User filter	Paginated audit log with timestamps
Reports Page	Date range selection	Charts, statistics, CSV export

5. Testing & Tools Used

5.1 Testing Strategy

The GPU Allocation System was tested through a comprehensive strategy encompassing unit testing of individual components, integration testing of API endpoints, and user acceptance testing simulating real-world usage scenarios.

5.2 Unit Testing

Individual React components were tested for correct rendering and state management. Backend utility functions including the cron job logic, date validation, and shift time calculation were tested with edge cases such as midnight boundary allocations, overlapping requests, and invalid input handling.

5.3 API Integration Testing

All RESTful API endpoints were tested using Postman. Test cases covered authentication flows (login, logout, signup, session validation), complete request lifecycle (create, approve, reject, release, cancel), extension request processing, GPU utilization queries, notification retrieval and deletion, and chat message operations.

5.4 Test Cases

Table 7: Key Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC-01	Valid student login	Valid @muj.manipal.edu email + password	Session created, redirect to student dashboard	PASS
TC-02	Invalid domain login	Non-MUJ email address	Error: 'Invalid email domain'	PASS
TC-03	Create GPU request	Valid form data with all required fields	Request created with PENDING status	PASS
TC-04	Admin approve request	Valid request ID, admin session	Status: APPROVED, GPU allocated, notification sent	PASS
TC-05	Admin reject request	Valid request ID with reject reason	Status: REJECTED, student notified	PASS
TC-06	Cron auto-release	Approved request past end time	Status: RELEASED, GPU freed, log created	PASS
TC-07	Extension approval	Chat message with extension request	End time updated, student notified	PASS
TC-08	Duplicate shift booking	Same GPU, same date, same shift	Error: 'Resource already allocated'	PASS

TC-09	Docker deployment	docker-compose up --build	Both containers running, app accessible	PASS
TC-10	Unauthorized access	Student accessing /api/admin routes	HTTP 403 Forbidden	PASS

5.5 Tools Used for Testing

- Postman: REST API endpoint testing and collection management
- Browser DevTools (Chrome): Frontend debugging, network request inspection, React component state inspection
- Docker Desktop: Container health monitoring and log inspection
- MongoDB Compass: Database query validation and data integrity verification
- ESLint: Static code analysis and TypeScript linting for code quality
- Vite Dev Server: Hot module replacement for rapid frontend development and testing

6. Implementation & Maintenance

6.1 Backend Implementation

The backend is implemented as a Node.js application using Express.js v5.1.0 with ES modules. The application follows a layered architecture with route handlers, controller functions, and Mongoose models. Session-based authentication uses express-session with MongoDB as the session store.

The cron job scheduler (node-cron v3.0.3) executes every minute to check for allocations past their end time. When expired allocations are detected, the system automatically updates request status to 'released', decrements GPU allocation records, recalculates utilization percentages, creates audit log entries, and dispatches student notifications.

6.2 Frontend Implementation

The frontend is a Single Page Application (SPA) built with React 18.3.1 and TypeScript 5.5.3, bundled with Vite v5.4.2. The application uses React Router DOM v7.9.4 for client-side navigation with protected routes that redirect unauthenticated users to the login page and enforce role-based access (students cannot access admin routes).

Tailwind CSS provides utility-first styling, enabling rapid UI development with consistent design tokens. The component architecture separates concerns between page components, layout components, and reusable UI components. Axios instances are configured with base URLs and interceptors for automatic session handling.

6.3 Deployment Implementation

The system is deployed using Docker with separate Dockerfiles for the frontend and backend. The frontend uses a multi-stage build: Stage 1 uses Node.js to run Vite build, Stage 2 copies the built artifacts to an Nginx Alpine image for efficient static file serving.

```
# Docker Compose Structure
services:
  backend:
    build: ./backend
    ports: ['5000:5000']
    environment: MONGO_URI, PORT, CLIENT_URL
  frontend:
    build: ./frontend
    ports: ['80:80']
    environment: VITE_API_BASE_URL
    depends_on: [backend]
```

6.4 API Endpoint Structure

Table 8: API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/login	User authentication	No
POST	/api/auth/signup	New user registration	No
POST	/api/auth/logout	Session termination	Yes
GET	/api/auth/admins	List admin users	Yes (Admin)
GET	/api/requests/	Get user's own requests	Yes (Student)
GET	/api/requests/all	Get all requests	Yes (Admin)
POST	/api/requests/	Create new GPU request	Yes (Student)
PUT	/api/requests/:id	Update request status	Yes (Admin)
PATCH	/api/requests/:id/extend	Request time extension	Yes
PATCH	/api/requests/:id/cancel	Cancel pending request	Yes (Student)
GET	/api/logs/	Get all allocation logs	Yes (Admin)

GET	/api/logs/user	Get user's logs	Yes (Student)
GET	/api/gpu/utilization	Get GPU utilization stats	Yes
GET	/api/notifications/	Get user notifications	Yes
DELETE	/api/notifications/:id	Delete notification	Yes
GET	/api/chat/	Get chat messages	Yes
POST	/api/chat/	Send chat message	Yes
PUT	/api/chat/:id/read	Mark message as read	Yes

6.5 Maintenance Considerations

The system is designed for ease of maintenance through several architectural decisions:

- **Environment-based configuration:** All sensitive values and environment-specific settings are externalized to .env files, allowing configuration changes without code modifications.
- **Docker containerization:** The containerized architecture enables consistent deployment, easy rollback to previous versions, and isolation of dependencies.
- **Structured logging:** All GPU operations create audit log entries with timestamps and action details, providing a complete history for debugging and compliance.
- **Dummy data mode:** A development mode with mock data allows frontend development and testing without requiring a live backend connection.
- **MongoDB Atlas compatibility:** The external MongoDB configuration supports cloud-hosted database services for production deployments, enabling automated backups and scaling.

7. Conclusion and Future Work

7.1 Conclusion

The GPU Allocation System successfully addresses the fundamental challenge of fair and efficient GPU resource management in academic computing environments. The project delivers a production-ready, full-stack MERN application that automates the complete GPU allocation lifecycle from request submission to automated release.

The system achieves its core objectives: preventing resource monopolization through priority-based scheduling and shift-based time slots, providing transparency through real-time dashboards

and comprehensive audit logs, reducing administrative overhead through automated cron-based release mechanisms, and ensuring institutional access control through email domain validation and role-based access control.

The technical implementation demonstrates modern software engineering best practices including containerized microservices architecture, TypeScript for type safety, RESTful API design, environment-based configuration, and separation of concerns across the full stack. The Docker containerization ensures deployment consistency and portability across development and production environments.

The project reinforces the importance of automated resource management systems in academic institutions where computational resources are shared among diverse users with varying needs and priorities. The platform's extensible architecture provides a solid foundation for the planned enhancements.

7.2 Future Work

Several enhancements are planned to extend the system's capabilities and integration:

- **LDAP/SSO Integration:** Replace the current email/password authentication with LDAP directory services or Single Sign-On (SSO) integration, enabling seamless login with university credentials and centralized user management.
- **Kubernetes & Slurm Integration:** Extend the system to manage GPU resources in Kubernetes clusters and integrate with Slurm workload manager for HPC environments, enabling actual GPU process scheduling rather than just time-slot booking.
- **Cloud GPU Support:** Add support for cloud GPU providers (AWS EC2, Google Cloud GCP, Azure) to dynamically provision additional GPU resources during peak demand periods.
- **CLI Utilities:** Develop command-line tools for administrators to manage allocations, generate reports, and perform bulk operations without requiring the web interface.
- **Advanced Analytics:** Implement predictive analytics for GPU demand forecasting, enabling proactive resource planning and policy adjustments based on historical usage patterns.

- **Mobile Application:** Develop a companion mobile application (React Native) for on-the-go request management and push notifications.
- **Queue Management:** Implement a request queue system with estimated wait times based on current allocation schedule and historical usage data.
- **Multi-Institution Support:** Extend the platform to support multiple institutions with separate resource pools, administrative domains, and institutional policies.

8. Outcome

The GPU Allocation System has been successfully implemented as a fully functional web application and deployed in a containerized environment. The primary outcome is a working platform that can be immediately deployed in academic lab environments to manage GPU resources.

Deployment Outcome: The system has been containerized using Docker and is deployable via a single docker-compose up command, demonstrating production-level deployment readiness. The frontend serves optimized static assets through Nginx with gzip compression, and the backend API handles concurrent requests with proper session management.

Academic Outcome: The project is a candidate for submission to technical conferences focused on educational technology and resource management systems. Documentation and source code are organized to support potential open-source publication, enabling adoption by other educational institutions facing similar GPU resource management challenges.

Skills Developed: The project involved mastery of full-stack web development with the MERN stack, containerization with Docker, TypeScript for large-scale frontend development, RESTful API design, MongoDB schema design and aggregation, cron-based task scheduling, role-based access control implementation, and Nginx configuration for production deployments.

9. Bibliography

The following references were used during the design, development, and documentation of the GPU Allocation System:

- [1] MongoDB Documentation. (2024). "MongoDB Manual v6.0." MongoDB Inc.
<https://docs.mongodb.com/> (Accessed: April 2025).
- [2] Chodorow, K. (2013). "MongoDB: The Definitive Guide." O'Reilly Media, Sebastopol, CA.
- [3] Banks, A. and Porcello, E. (2020). "Learning React: Modern Patterns for Developing React Apps." 2nd ed. O'Reilly Media, Sebastopol, CA.
- [4] Cherny, B. (2019). "Programming TypeScript: Making Your JavaScript Applications Scale." O'Reilly Media, Sebastopol, CA.
- [5] Hahn, E. (2012). "Express.js Guide: The Comprehensive Book on Express.js." Azat Mardan.
- [6] Kane, S. and Matthias, K. (2018). "Docker: Up and Running." 2nd ed. O'Reilly Media, Sebastopol, CA.
- [7] React Documentation. (2024). "React 18 Official Documentation." Meta Open Source.
<https://react.dev/> (Accessed: April 2025).
- [8] Node.js Foundation. (2024). "Node.js v18 API Documentation." OpenJS Foundation.
<https://nodejs.org/docs/> (Accessed: April 2025).
- [9] Tailwind Labs. (2024). "Tailwind CSS Documentation v3." <https://tailwindcss.com/docs> (Accessed: April 2025).
- [10] Richardson, L. and Amundsen, M. (2013). "RESTful Web APIs." O'Reilly Media, Sebastopol, CA.
- [11] Nginx Inc. (2024). "Nginx Documentation." <https://nginx.org/en/docs/> (Accessed: April 2025).
- [12] Mongoose. (2024). "Mongoose v8 Documentation." <https://mongoosejs.com/docs/> (Accessed: April 2025).